#AI Assisted Coding

#Assignment –17.1

#V.Pranavith || 2303A51488 || Batch:8

```
# Generate a clean and simple Python script using pandas to preprocess
a customer dataset (shopping_trends.csv).

# Tasks:
# - Handle missing values in age, income, and city columns
# - Remove duplicate rows
# - Convert city names to lowercase
# - Encode categorical columns (gender, city) using label encoding or
one-hot encoding

# Keep the code beginner-friendly and well-structured. Output the
cleaned dataframe.

import pandas as pd from sklearn.preprocessing import
LabelEncoder, OneHotEncoder from sklearn.compose import
ColumnTransformer from sklearn.impute import SimpleImputer
import numpy as np

# Load the dataset try:
    df =
    pd.read_csv('shopping_trends.csv')
    print("Original DataFrame head:")
    print(df.head())  print("\nOriginal
    DataFrame info:") df.info()
except FileNotFoundError:
    print("Error: 'shopping_trends.csv' not found. Please ensure
the  file is uploaded.") exit()

# --- 1. Handle missing values ---

# Fill missing 'Age' with the mean
df['Age'] = df['Age'].fillna(df['Age'].mean())

# Fill missing 'Purchase Amount (USD)' with the mean
df['Purchase Amount (USD)'] = df['Purchase Amount
(USD)'].fillna(df['Purchase Amount (USD)'].mean())

# For 'Location', fill missing values with the mode
# First, check if there are any missing values before trying to fill if
df['Location'].isnull().any():
    mode_city = df['Location'].mode()[0]
```

```
    df['Location'] = df['Location'].fillna(mode_city)

print("\nDataFrame after handling missing values:")
df.info()

# --- 2. Remove duplicate rows --- df.drop_duplicates(inplace=True)
print(f"\nDataFrame after removing duplicates. New shape: {df.shape}")

# --- 3. Convert city names to lowercase --- df['Location'] =
df['Location'].str.lower() print("\nDataFrame head after
converting city names to lowercase:")
print(df[['Location']].head())

# --- 4. Encode categorical columns ---

# Label Encoding for 'Gender' label_encoder
= LabelEncoder()  df['Gender_Encoded'] =
label_encoder.fit_transform(df['Gender']) print("\nDataFrame
head after Label Encoding 'Gender':") print(df[['Gender',
'Gender_Encoded']].head())

# One-Hot Encoding for 'Location'  one_hot_encoder =
OneHotEncoder(handle_unknown='ignore',
sparse_output=False)

# Reshape the 'Location' column to a 2D array for OneHotEncoder
cities_encoded = one_hot_encoder.fit_transform(df[['Location']])

# Create a DataFrame from the one-hot encoded cities cities_df
= pd.DataFrame(cities_encoded,
columns=one_hot_encoder.get_feature_names_out(['Location']))

# Reset index for original df to ensure proper concatenation
df.reset_index(drop=True, inplace=True)

# Concatenate the one-hot encoded city DataFrame with the
original DataFrame  df = pd.concat([df, cities_df], axis=1)

# Drop the original 'Gender' and 'Location' columns as they are now
encoded
df.drop(['Gender', 'Location'], axis=1, inplace=True)

print("\nCleaned DataFrame head with encoded columns:")
print(df.head())

print("\nCleaned DataFrame info:") df.info()
```

```
Original DataFrame head:
   Customer ID  Age Gender Item Purchased  Category  Purchase Amount
(USD)  \
0            1   55   Male         Blouse  Clothing
53
1            2   19   Male        Sweater  Clothing
64
2            3   50   Male          Jeans  Clothing
73
3            4   21   Male        Sandals  Footwear
90
4            5   45   Male         Blouse  Clothing
49
                                          Location Size       Color
Season Review Rating Subscription
Status \

0           Kentucky  L   Gray Winter                 3.1

Yes

1           Maine  L      Maroon Winter              3.1
Yes

2 Massachusetts      S     Maroon Spring              3.1
Yes

3 Rhode Island       M     Maroon Spring              3.5
Yes

4         Oregon     M Turquoise Spring              2.7
Yes
   Payment Method Shipping Type Discount Applied Promo Code Used \
0 Credit Card Express Yes Yes 1 Bank Transfer Express Yes Yes 2
Cash Free Shipping Yes Yes
3            PayPal Next Day Air      Yes  Yes
4            Cash Free Shipping  Yes  Yes
 Previous Purchases Preferred Payment Method Frequency of Purchases
0                   14     Venmo      Fortnightl
                                      y
1                   2      Cash
                           Fortnightly
2                   23     Credit Card     Weekl
                                           y
3                   49     PayPal     Weekl
                                      y

4                   31     PayPal     Annually
Original DataFrame info:
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
```

```
# Column                          Non-Null Count Dtype

 0 Customer ID 3900 non-null int64 1 Age 3900 non-null int64
                                                          2     Gender  3900
 4 Category  3900 non-null object                         non-null object
5 Purchase Amount (USD)           3900 non-null int64     3     Item
6 Location  3900 non-null object 7 Size           3900    Purchased  3900
 Color  3900 non-null object                              non-null object


                                                          non-null object 8

 9   Season  3900 non-null object
 10  Review Rating  3900 non-null float64
 11  Subscription Status  3900 non-null object
 12  Payment Method  3900 non-null object 13 Shipping Type  3900
 nonnull object 14 Discount Applied   3900 non-null object
  15 Promo Code Used  3900 non-null object
  16 Previous Purchases   3900 non-null int64
  17 Preferred Payment Method 3900 non-null object
  18 Frequency of Purchases 3900 non-null object             dtypes:
                                                          float64(1),
 int64(4), object(14) memory usage: 579.0+ KB
DataFrame after handling missin
values:
<class
'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to
3899
Data columns (total 19 columns):
# Column                      Non-NullCount Dtype
```

```
                                                       int64
 0   Customer ID     3900 non-  null null  int64
 1   Age  3900 non- null  null  object 2 Gender  3900 non-
 null  object
 3   Item Purchased  3900 non-  object
 4   Category  3900 non-
 5   Purchase Amount (USD)       3900 non-null   int64
 6 Location  3900 non- null   objec t 7 Size  3900 non- null
 objec t  8 Color  3900 non- null    objec t 9 Season  3900
 non- null    objec t
10 Review Rating            3900 non-null         float 64
```

```
 11 Subscription Status 3900 non12null    objec t Payment
 Method 3900 non13 Shippingnull   objec t
Type 3900 non-                          null         objec t
 14  Discount Applied      3900 non- null        objec t
 15  Promo Code Used  3900 non- null        objec t
16 Previous Purchases        3900 non-null int64
 17  Preferred Payment Method 3900 non-null object
 18  Frequency of Purchases      3900 non-null object
dtypes:    float64(1),      int64(4),
object(14)
```

```
memory usage: 579.0+ KB

DataFrame after removing duplicates. New shape: (3900, 19)
DataFrame head after converting city names to lowercase:
                Location
0          kentucky
1          maine
2          massachusetts
3          rhode island
4          oregon
DataFrame head after Label Encoding 'Gender':
  Gender  Gender_Encoded
0   Male               1
1   Male               1
2   Male               1
3   Male               1
4   Male               1

Cleaned DataFrame head with encoded columns:
     Customer ID Age Item Purchased Category Purchase Amount (USD)
Size \                                                                0
              1 55          Blouse Clothing                     53 L
                                                                    1
                                                                  L
              2 19         Sweater Clothing                     64   2
                                                                  S
              3 50           Jeans Clothing                     73   3
                                                                  M
              4 21         Sandals Footwear                     90   4
                                                                  M

              5 45          Blouse Clothing                     49

         Color Season Review Rating Subscription Status ..............\
0       Gray Winter             3.1                  Yes ...
1     Maroon Winter             3.1                  Yes ...
2     Maroon Spring             3.1                  Yes ...
3     Maroon Spring             3.5                  Yes ...
4  Turquoise Spring             2.7                  Yes ...
            Location_south dakota Location_tennessee Location_texas
Location_utah \

0                     0.0                 0.0             0.0
0.0
1                     0.0                 0.0             0.0


0.0

                      0.0                 0.0             0.0
```

```
2

0.0

3                    0.0                 0.0             0.0
0.0
4                    0.0                 0.0             0.0
0.0

   Location_vermont Location_virginia Location_washington \
0               0.0   0.0  0.0
1               0.0   0.0  0.0
2               0.0   0.0  0.0
3               0.0   0.0  0.0
4               0.0   0.0  0.0

     Location_west virginia Location_wisconsin Location_wyoming
0                     0.0  0.0  0.0
1                     0.0  0.0  0.0
2                     0.0  0.0  0.0
3                     0.0  0.0  0.0
4                     0.0  0.0  0.0

[5 rows x 68 columns]

Cleaned DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 68 columns):
  # Column                      Non-Null Count Dtype




 0   Customer ID  3900 non-null int64
 1   Age   3900 non-null int64
 2   Item Purchased   3900 non-null object
 3   Category   3900 non-null object
 4   Purchase Amount (USD)  3900 non-null int64
 5   Size  3900 non-null object
 6   Color  3900 non-null object
 7   Season  3900 non-null object
 21  Location_arkansas  3900 non-null float64
```

```
8 Review Rating  3900 non-null float64                ment Method 3900
9 Subscription Status 3900 non-null object   10 Pay
    non-null object
11  Shipping Type  3900 non-null object
12  Discount Applied  3900 non-null object
13 Promo Code Used  3900 non-null object
14  Previous Purchases  3900 non-null int64
15  Preferred Payment Method 3900 non-null object

16  Frequency of Purchases  3900 non-null object
17  Gender_Encoded   3900 non-null int64
18  Location_alabama  3900 non-null float64
19  Location_alaska  3900 non-null float64
20  Location_arizona  3900 non-null float64
22  Location_california   3900 non-null float64
23  Location_colorado  3900 non-null float64
```

```
 24 Location_connecticut       3900 non-null   float64
 25 Location_delaware          3900 non-null   float64
 26 Location_florida           3900 non-null   float64
 27 Location_georgia           3900 non-null   float64
 28 Location_hawaii            3900 non-null   float64

 29 Location_idaho             3900 non-null   float64
 30 Location_illinois          3900 non-null   float64
 31 Location_indiana           3900 non-null   float64
 32 Location_iowa              3900 non-null   float64
 33 Location_kansas            3900 non-null   float64
 34 Location_kentucky          3900 non-null   float64
 35 Location_louisiana         3900 non-null   float64
 36 Location_maine             3900 non-null   float64
 37 Location_maryland          3900 non-null   float64
 38 Location_massachusetts     3900 non-null   float64

 39 Location_michigan          3900 non-null   float64
 40 Location_minnesota         3900 non-null   float64
 41 Location_mississippi       3900 non-null   float64
```

```
 42 Location_missouri          3900 non-null   float64
 43 Location_montana           3900 non-null   float64
 44 Location_nebraska          3900 non-null   float64
 45 Location_nevada            3900 non-null   float64
 46 Location_new hampshire     3900 non-null   float64
 47 Location_new jersey        3900 non-null   float64
 48 Location_new mexico        3900 non-null   float64
 49 Location_new york          3900 non-null   float64
 50 Location_north carolina    3900 non-null   float64
 51 Location_north dakota      3900 non-null   float64
 52 Location_ohio              3900 non-null   float64
 53 Location_oklahoma          3900 non-null   float64
 54 Location_oregon            3900 non-null   float64
 55 Location_pennsylvania      3900 non-null   float64
 56 Location_rhode island      3900 non-null   float64
 57 Location_south carolina    3900 non-null   float64
 58 Location_south dakota      3900 non-null   float64
 59 Location_tennessee         3900 non-null   float64
 60 Location_texas             3900 non-null   float64
 61 Location_utah              3900 non-null   float64
 62 Location_vermont           3900 non-null   float64
 63 Location_virginia          3900 non-null   float64
 64 Location_washington        3900 non-null   float64
 65 Location_west virginia     3900 non-null   float64
 66 Location_wisconsin         3900 non-null   float64
 67 Location_wyoming           3900 non-null   float64
dtypes: float64(51), int64(5), object(12) memory
usage: 2.0+ MB
```

```
import pandas as pd from sklearn.preprocessing import MinMaxScaler,
StandardScaler import numpy as np

# Load the dataset try:
    df_sales = pd.read_csv('sales_data.csv') print("Original
    Sales DataFrame head:") print(df_sales.head())
    print("\nOriginal
    Sales DataFrame info:") df_sales.info()
except FileNotFoundError:
    print("Error: 'sales_data.csv' not found. Please ensure the file
is uploaded.") exit()

# --- 1. Convert date column to datetime format ---
# Assuming the date column is named 'Sale_Date' if
'Sale_Date' in df_sales.columns:
    df_sales['Sale_Date'] = pd.to_datetime(df_sales['Sale_Date'],
errors='coerce')
    # Drop rows where 'Sale_Date' could not be parsed
    df_sales.dropna(subset=['Sale_Date'], inplace=True)
    print("\nDataFrame after converting 'Sale_Date' to datetime:")
    df_sales.info()
else:
    print("Warning: 'Sale_Date' column not found. Skipping date
conversion.")

# --- 2. Extract new features: month, year, day of week --- if
'Sale_Date' in df_sales.columns:
    df_sales['Sale_Month'] = df_sales['Sale_Date'].dt.month
    df_sales['Sale_Year'] = df_sales['Sale_Date'].dt.year
    df_sales['Sale_DayOfWeek'] = df_sales['Sale_Date'].dt.dayofweek #
Monday=0, Sunday=6 print("\nDataFrame head after extracting date
features:") print(df_sales[['Sale_Date', 'Sale_Month', 'Sale_Year',
'Sale_DayOfWeek']].head())

# --- 3. Detect and handle outliers (using IQR method for
'Sales_Amount') ---
# Assuming the sales amount column is named 'Sales_Amount' if
'Sales_Amount' in df_sales.columns:
    Q1 = df_sales['Sales_Amount'].quantile(0.25)
    Q3 = df_sales['Sales_Amount'].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR upper_bound
    = Q3 + 1.5 * IQR
```

```
    # Identify outliers outliers =
df_sales[(df_sales['Sales_Amount'] < lower_bound) |
(df_sales['Sales_Amount'] > upper_bound)] print(f"\nNumber of
outliers detected in 'Sales_Amount': {len(outliers)}")
```

```
    # Handle outliers by capping them at the bounds
    df_sales['Sales_Amount_Capped'] =
df_sales['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)
    print("DataFrame head after capping 'Sales_Amount' outliers:")
    print(df_sales[['Sales_Amount', 'Sales_Amount_Capped']].head())
else:
    print("Warning: 'Sales_Amount' column not found. Skipping outlier
detection and handling.")

# --- 4. Normalize the sales amount column using StandardScaler ---
# Using the capped Sales_Amount for normalization if
'Sales_Amount_Capped'     in     df_sales.columns:     scaler     =
StandardScaler()
df_sales['Sales_Amount_Normalized'] =
scaler.fit_transform(df_sales[['Sales_Amount_Capped']])
print("\nDataFrame head after normalizing 'Sales_Amount_Capped':")
print(df_sales[['Sales_Amount_Capped',
'Sales_Amount_Normalized']].head()) else:
    print("Warning: 'Sales_Amount_Capped' column not found. Skipping
normalization.")

print("\nTransformed DataFrame head:")
print(df_sales.head()) print("\nTransformed
DataFrame info:") df_sales.info()
```

```
Original Sales DataFrame head:
 Product_ID Sale_Date Sales_Rep Region Sales_Amount  Quantity_Sold \
0        1052 2023-02-03  Bob North  5053.97
 18
1        1093 2023-04-21  Bob West  4384.02
 17
2        1015 2023-09-21  David South    4631.23
 30
3        1072 2023-08-24  Bob South  2167.94
 39
4        1061 2023-03-24 Charlie East    3750.20
 13

 Product_Category Unit_Cost Unit_Price Customer_Type  Discount \
0             Furniture  152.75  267.22  Returning  0.09
1             Furniture  3816.39  4209.44  Returning  0.11


2             Food    261.56  371.40  Returning  0.20
3             Clothing  4330.03  4467.75  New  0.02
4             Electronics  637.37  692.71  New  0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep
0 Cash  Online  North-Bob 1      Cash  Retail  West-Bob
```

```
2    Bank Transfer Retail     South-David
3    Credit Card   Retail     South-Bob
4    Credit Card   Online     East-Charlie

Original Sales DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
  # Column                  Non-Null Count Dtype
---  ------                  --------------  -----
  0     Product_ID      1000 non-null int64
  1     Sale_Date  1000 non-null object
  2     Sales_Rep  1000 non-null object
  3     Region     1000 non-null object
  4     Sales_Amount    1000 non-null float64
  5     Quantity_Sold   1000 non-null int64
  6     Product_Category      1000 non-null object
  7     Unit_Cost  1000 non-null float64
  8     Unit_Price 1000 non-null float64   9 Customer_Type 1000 non-
 null object
 Discount    1000 non-null float64
 Payment_Method    1000 non-null object
 Sales_Channel     1000 non-null object
 Region_and_Sales_Rep  1000  non-null  object  dtypes:  float64(4),
 int64(2), object(8) memory usage: 109.5+ KB

DataFrame after converting 'Sale_Date' to datetime:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
  # Column                  Non-Null Count Dtype
---  ------                  --------------  -----
```

```
0   Product_ID   1000 non-null int64
1   Sale_Date    1000 non-null datetime64[ns]
2   Sales_Rep    1000 non-null object
3   Region  1000 non-null object
4   Sales_Amount  1000 non-null float64
5   Quantity_Sold  1000 non-null int64
6   Product_Category  1000 non-null object
7   Unit_Cost    1000 non-null float64
8   Unit_Price   1000 non-null float64
9   Customer_Type  1000 non-null object
```

```
 10  Discount    1000 non-null float64
 11  Payment_Method   1000 non-null object
 12  Sales_Channel    1000 non-null object
 13  Region_and_Sales_Rep 1000 non-null object
dtypes: datetime64[ns](1), float64(4), int64(2), object(7)
memory usage: 109.5+ KB
DataFrame  head  after  extracting  date  features:
   Sale_Date Sale_Month Sale_Year Sale_DayOfWeek
0 2023-02-03     2     2023  4
1 2023-04-21     4     2023  4
2 2023-09-21            9       2023                3
3 2023-08-24            8       2023                3
4 2023-03-24            3       2023                4
Number of outliers detected in 'Sales_Amount': 0
DataFrame   head   after   capping   'Sales_Amount'
outliers:
   Sales_Amount Sales_Amoun _Capped
0       5053.97             5053.97
1       4384.02             4384.02

2       4631.23             4631.23

3       2167.94             2167.94

4       3750.20             3750.20

DataFrame        head        after        normalizing
'Sales_Amount_Capped':
   Sales_Amount_Capped Sales_Amount_Normalized
0              5053.97                0.012197
1              4384.02               -
                                     0.223256
2              4631.23               -
                                     0.136374
3              2167.94               -
                                     1.002094
4              3750.20               -
                                     0.446011
Transformed DataFrame head:
 Product_ID Sale_Date Sales_Rep Region Sales_Amount Quantity_Sold \
     0     1052 2023-02-03  Bob North  5053.97    18

1          10932023-04-21       Bob   West       4384.02              17


2          10152023-09-21     David  South       4631.23              30
```

```
3          10722023 -08-24      Bob  South       2167.94             39

4          10612023 -03-24  Charlie  East        3750.20             13
      Product_Category Unit_Cost Unit_Price Customer_Type Discount \
0        Furniture     152.75      267.22     Returning      0.09
1         Furniture  3816.39      4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
```

```
3          Clothing  4330.03  4467.75  New  0.02
4          Electronics  637.37  692.71  New  0.08
```

```
            Payment_Method Sales_Channel Region_and_Sales_Rep Sale_Month
 Sale_Year \
 0           Cash  Online  North-Bob  2
2023
 1           Cash  Retail  West-Bob  4
2023
 2           Bank Transfer  Retail  South-David   9
2023
 3           Credit Card  Retail  South-Bob  8
2023
 4           Credit Card  Online  East-Charlie  3
2023


 Sale_DayOfWeek Sales_Amount_Capped Sales_Amount_Normalized
 0               4  5053.97  0.012197
 1               4  4384.02  -0.223256
 2               3  4631.23  -0.136374
 3               3  2167.94  -1.002094
 4               4  3750.20  -0.446011

 Transformed DataFrame info:
 <class 'pandas.core.frame.DataFrame'>
 RangeIndex: 1000 entries, 0 to 999
 Data columns (total 19 columns):
   # Column                     Non-Null Count Dtype



      0     Product_ID      1000 non-null int64
      1     Sale_Date  1000 non-null datetime64[ns]
      2     Sales_Rep  1000 non-null object
      3     Region  1000 non-null object
      4     Sales_Amount  1000 non-null float64
      5     Quantity_Sold  1000 non-null int64
      6     Product_Category 1000 non-null object  7 Unit_Cost 1000 nonnull
    float64
8 Unit_Price 1000 non-null float64  9 Customer_Type 1000 non-null
 object
   10     Discount  1000 non-null float64
```

```
 11     Payment_Method  1000 non-null object
 12     Sales_Channel   1000 non-null object
 13     Region_and_Sales_Rep  1000 non-null object
 14     Sale_Month      1000 non-null int32
 15     Sale_Year  1000 non-null int32
 16     Sale_DayOfWeek  1000 non-null int32
 17     Sales_Amount_Capped     1000     non-null     float64     18
Sales_Amount_Normalized     1000     non-null     float64     dtypes:
datetime64[ns](1), float64(6), int32(3), int64(2), object(7) memory
usage: 136.8+ KB
```

```
import pandas as pd  from sklearn.model_selection
import train_test_split  from sklearn.preprocessing
import LabelEncoder,
OneHotEncoder, StandardScaler from sklearn.impute import
SimpleImputer from sklearn.compose import ColumnTransformer from
sklearn.pipeline import Pipeline import numpy as np

# Load the dataset try:
    # Create a dummy dataset for demonstration purposes if the file is
not found.
    # In a real scenario, you would replace this with
pd.read_csv('healthcare_dataset.csv').
    data = {
        'PatientID': range(1, 101),
        'Age': np.random.randint(20, 80, 100),
        'Gender': np.random.choice(['Male', 'Female', 'Other'], 100),
        'BloodPressure': np.random.normal(120, 15, 100).round(2),
        'Cholesterol': np.random.normal(200, 30, 100).round(2),
        'BMI': np.random.normal(25, 5, 100).round(2),
        'SmokingStatus': np.random.choice(['Yes', 'No'], 100),
        'MedicalHistory': np.random.choice(['None', 'Diabetes', 'Heart
Disease'], 100),
        'DiseaseOutcome': np.random.choice([0, 1], 100, p=[0.7, 0.3]) #
Target variable (0 or 1)
    }
    df = pd.DataFrame(data)

    # Introduce some missing values for demonstration
    np.random.seed(42) # for reproducibility of missing values
    for col in ['BloodPressure', 'Cholesterol', 'BMI']:
        missing_indices = np.random.choice(df.index, 10,
replace=False) df.loc[missing_indices, col] =
        np.nan

    print("Original DataFrame head:")
    print(df.head()) print("\nOriginal DataFrame
    info:") df.info()

except FileNotFoundError: print("Error: 'healthcare_dataset.csv'
    not found. Please ensure
the file is uploaded.") print("Generating a dummy dataset for
    demonstration as the file
was not found.")
    # If the file is not found, the dummy data created above will be
used.
```

```
# Define target variable and features
target_column = 'DiseaseOutcome'
X = df.drop(columns=[target_column, 'PatientID']) # Drop identifier
column and target for features y = df[target_column]

# Identify numerical and categorical features numerical_features =
X.select_dtypes(include=np.number).columns.tolist()
categorical_features =
X.select_dtypes(include='object').columns.tolist()

print(f"\nFeatures identified:") print(f"Numerical features:
{numerical_features}") print(f"Categorical features:
{categorical_features}")

# --- Create preprocessing pipelines ---

#  Pipeline  for  numerical  features:  impute  missing  values  then  scale
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')), # Handles missing
values in numerical columns
 ('scaler', StandardScaler())  # Scales numerical  features
])

#    Pipeline   for   categorical     features:  one-hot    encode
categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # Encodes
categorical features ])

# Create a ColumnTransformer to apply different transformations to
different columns
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ],
    remainder='passthrough' # Keep any other columns not specified (if
any) as they are )

# Apply preprocessing to the features (X)
X_processed = preprocessor.fit_transform(X)

# Get the names of the processed columns
# Numerical features retain their names after scaling #
OneHotEncoder creates new names for categorical features
onehot_feature_names = preprocessor.named_transformers_['cat']
```

```
['onehot'].get_feature_names_out(categorical_features)
processed_feature_names = numerical_features +
list(onehot_feature_names)

# Convert the processed array back to a DataFrame for better
readability and future use
X_processed_df = pd.DataFrame(X_processed,
columns=processed_feature_names)

print("\nProcessed Features (X_processed_df)
head:") print(X_processed_df.head())
print("\nProcessed Features (X_processed_df)
info:") X_processed_df.info()

# --- Handle the target variable (y) ---
# If y is categorical (e.g., 'Yes'/'No'), it needs to be encoded.
# If it's already numerical (0/1), we can use it as is.
if y.dtype == 'object':   le_y = LabelEncoder() y_processed =
    le_y.fit_transform(y)  print("\nTarget variable 'y' encoded
    to numerical values.")
else:
    y_processed = y
    print("\nTarget variable 'y' is already numerical.")

# --- Split dataset into training and testing sets --- # Using
stratify=y_processed to ensure class distribution is maintained in
splits
X_train, X_test, y_train, y_test = train_test_split(X_processed_df,
y_processed, test_size=0.2, random_state=42, stratify=y_processed)

print("\nDataset split into training and testing sets:")
print(f"X_train shape: {X_train.shape}") print(f"X_test shape:
     {X_test.shape}")      print(f"y_train shape:
{y_train.shape}") print(f"y_test shape: {y_test.shape}")

print("\nX_train head (first 5 rows):") print(X_train.head())
print("\nX_test head (first 5 rows):")
print(X_test.head()) print("\ny_train head
(first 5 values):") print(y_train.head())
print("\ny_test head (first 5 values):")
print(y_test.head())
```

```
Original DataFrame head:

                          PatientID Age Gender BloodPressure
Cholesterol             BMI
SmokingStatus \

0           1 65     Male             NaN        220.56 27.64
```

```
Yes
1           2   34    Male         142.35       118.47  21.44
No
2           3   54    Male          97.53       201.93  22.47
Yes
3           4   34  Female         119.46       213.83  23.14
Yes
4           5   54    Male         132.58       164.31  22.67
Yes

  MedicalHistory  DiseaseOutcome
0           None               1
1  Heart Disease               1
2  Heart Disease               1
3           None               0
4       Diabetes               0

Original DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   PatientID        100 non-null    int64
 1   Age              100 non-null    int64
 2   Gender           100 non-null    object
 3   BloodPressure    90 non-null     float64
 4   Cholesterol      90 non-null     float64
 5   BMI              90 non-null     float64
 6   SmokingStatus    100 non-null    object
 7   MedicalHistory   100 non-null    object
 8   DiseaseOutcome   100 non-null    int64
dtypes: float64(3), int64(3), object(3)
memory usage: 7.2+ KB

Features identified:
Numerical features: ['Age', 'BloodPressure', 'Cholesterol', 'BMI']
Categorical features: ['Gender', 'SmokingStatus', 'MedicalHistory']

Processed Features (X_processed_df) head:
        Age  BloodPressure  Cholesterol         BMI  Gender_Female
Gender_Male  \
0  0.863002   9.888531e-16     0.647241  0.441449            0.0
1.0
1 -1.035725   1.442507e+00    -2.688063 -0.888691            0.0
1.0
2  0.189260  -1.676264e+00     0.038594 -0.667716            0.0
1.0
3 -1.035725  -1.502790e-01     0.427370 -0.523975            1.0
0.0
```

```
4  0.189260   7.626676e-01    -1.190460 -0.624808                0.0
1.0

   Gender_Other  SmokingStatus_No  SmokingStatus_Yes
MedicalHistory_Diabetes  \
0           0.0                0.0                1.0
0.0
1           0.0                1.0                0.0
0.0
2           0.0                0.0                1.0
0.0
3           0.0                0.0                1.0
0.0
4           0.0                0.0                1.0
1.0

   MedicalHistory_Heart  Disease   MedicalHistory_None
0                            0.0                   1.0
1                            1.0                   0.0
2                            1.0                   0.0
3                            0.0                   1.0
4                            0.0                   0.0

Processed Features (X_processed_df) info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 12 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Age                             100 non-null    float64
 1   BloodPressure                   100 non-null    float64
 2   Cholesterol                     100 non-null    float64
 3   BMI                             100 non-null    float64
 4   Gender_Female                   100 non-null    float64
 5   Gender_Male                     100 non-null    float64
 6   Gender_Other                    100 non-null    float64
 7   SmokingStatus_No                100 non-null    float64
 8   SmokingStatus_Yes               100 non-null    float64
 9   MedicalHistory_Diabetes         100 non-null    float64
 10  MedicalHistory_Heart Disease    100 non-null    float64
 11  MedicalHistory_None             100 non-null    float64
dtypes: float64(12)
memory usage: 9.5 KB

Target variable 'y' is already numerical.

Dataset split into training and testing sets:
X_train shape: (80, 12)
X_test shape: (20, 12)
y_train shape: (80,)
```

```
y_test shape: (20,)
X_train head (first 5 rows):
          Age BloodPressure Cholesterol         BMI Gender_Female \
   -1.586969 -9.948242e-02       0.000000 0.181857             0.0    25
  -0.729479 4.690217e-01       -0.876499 -0.474631             0.0    8
 38  -0.729479                  -0.758559 1.108665             0.0
              -8.997024e-01

 70  -1.709467
         9.888531e-16  -0.464527 -1.839098                     0.0
58 1.169249 1.125202e+00       -0.312611 -0.575465             0.0
    Gender_Male Gender_Other SmokingStatus_No SmokingStatus_Yes \
25         0.0       1.0   1.0   0.0 0.0    1.0   1.0   0.0 0.0    1.0
8                          1.0   0.0 0.0    1.0   0.0   1.0
38          1.0            0.0                1.0                 0.0
70
58

    MedicalHistory_Diabetes MedicalHistory_Heart Disease

Med calHistory_None

25                      0.0                            1.0
0.0

8                       0.0                            0.0
1.0

38                      1.0                            0.0
0.0

70                      0.0                            1.0
0.0

58                      0.0                            1.0
0.0

X_test head (first 5 rows):
          Age BloodPressure Cholesterol         BMI Gender_Female \
95 -0.361983        -0.674249  0.405808 -0.566883             0.0
52 -1.586969        -1.233011  1.573115 -0.054135             1.0
30 -0.423232        -1.118197  1.039610 -0.133515             1.0
   0.250510           1.214270 -0.081959 0.123932             1.0
46
```

```
4 0.189260           0.762668 -1.190460 -0.624808              0.0
```

```
 Gender_Male Gender_Other SmokingStatus_No SmokingStatus_Yes \
95          0.0           1.0               0.0                1.0
52          0.0           0.0               1.0                0.0
30          0.0           0.0               0.0                1.0
46          0.0           0.0               0.0                1.0
4           1.0           0.0               0.0                1.0
```

```
   MedicalHistory_Diabetes  MedicalHistory_Heart Disease
```

```
MedicalHistory_None
95                          1.0                            0.0
0.0

52                          0.0                            0.0
```

```
1.0                                                                    stration ---

30                        0.0                             0.0
1.0

46                        0.0                             1.0
0.0

4                         1.0                             0.0
```

```
0.0
y_train head (first 5 values):
25       0
8        0

38       0
70       0
58       0
Name:     iseaseOutcome, dtype: int64

y_test    head (first 5 values):
95       0
52       1
30       0
46       1
4        0
Name: DiseaseOutcome, dtype: int64
import   pandas   as   pd      from
sklearn.impute                import
sklearn.preprocessing import np
```

```
),
    , 'IT',
 ndon',

,
 (2),
```

```
SimpleImputer from
Encoder import numpy as

# --- 1. Create a dummy employee salar
          # Attempt to load an existin
                  pd.read_csv('employ
    'employee_salary_data.csv'")
except FileNotFoundError:
    # Create a dummy dataset if the
    dummy 'employee_salary_data.csv' f
demonstration.") data = {
        'EmployeeID': range(1, 101
        'Age': np.random.randint(
        'Gender': np.random.choice
        'Department': np.random.choice
'HR', 'Finance'], 100),
        'Job_Location': np.random.choi
'Paris', 'Berlin'], 100),
        'Years_Experience':
        'Salary':    np.random.normal
    'Performance_Score': np.random.ran
```

```
dataset for try:
            df_salary =
  )         print("Loaded

print("Creating

),
2, 60, 100),
(['Male', 'Female'], 100
    (['Sales', 'Marketing'

    (['New York',

dom.randint(0, 30, 100)
    50000,    15000,    100).roun
        (1, 5, 100) }
```

```
    df_salary = pd.DataFrame(data)

    # Introduce some missing values np.random.seed(42) for col in
    ['Age', 'Department', 'Salary', 'Years_Experience']:
        missing_indices = np.random.choice(df_salary.index, 5,
replace=False) df_salary.loc[missing_indices, col]
        = np.nan

    # Introduce some outliers in Salary
    df_salary.loc[5, 'Salary'] = 150000.00 # High outlier
    df_salary.loc[10, 'Salary'] = 10000.00 # Low outlier
    df_salary.to_csv('employee_salary_data.csv',
    index=False)

print("Original DataFrame head:") print(df_salary.head())
print("\nOriginal DataFrame info:") df_salary.info()

# --- 2. Handle missing values in all columns ---

# Separate numerical and categorical columns for imputation
numerical_cols =
df_salary.select_dtypes(include=np.number).columns.tolist()
categorical_cols =
df_salary.select_dtypes(include='object').columns.tolist()

# Impute numerical columns with the mean for col
in numerical_cols:
    if df_salary[col].isnull().any():
        df_salary[col] = df_salary[col].fillna(df_salary[col].mean())

# Impute categorical columns with the most frequent value (mode) for
col in categorical_cols:
    if df_salary[col].isnull().any():
        df_salary[col] = df_salary[col].fillna(df_salary[col].mode()
[0])

print("\nDataFrame after handling missing values:") df_salary.info()

# --- 3. Detect and remove outliers in the 'Salary' column (using IQR)
--- if 'Salary' in df_salary.columns:
    Q1 = df_salary['Salary'].quantile(0.25)
    Q3 = df_salary['Salary'].quantile(0.75)
    IQR = Q3 - Q1 lower_bound = Q1 - 1.5 *
    IQR
```

```
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers
    outliers = df_salary[(df_salary['Salary'] < lower_bound) |
(df_salary['Salary'] > upper_bound)] print(f"\nNumber of
    outliers detected in 'Salary':
{len(outliers)}") print("Outliers in Salary:\n",
    outliers[['EmployeeID', 'Salary']])

    # Remove outliers
    df_salary_cleaned = df_salary[~((df_salary['Salary'] <
lower_bound) | (df_salary['Salary'] > upper_bound))] print(f"\nDataFrame
    shape after removing Salary outliers:
{df_salary_cleaned.shape}")
    # Update df_salary to the cleaned version df_salary
    = df_salary_cleaned.copy()  else:
    print("\n'Salary' column not found, skipping outlier detection.")

# --- 4. Convert department names to lowercase --- if
'Department' in df_salary.columns:
    df_salary['Department']    =    df_salary['Department'].str.lower()
    print("\nDataFrame head after converting 'Department' to
lowercase:") print(df_salary[['Department']].head())
else:
    print("\n'Department' column not found, skipping lowercase
conversion.")

# --- 5. Apply label encoding to categorical columns (Job_Location,
Department) --- label_encoder = LabelEncoder()

categorical_cols_to_encode = ['Job_Location', 'Department'] # Specify
columns for label encoding

for col in categorical_cols_to_encode:
    if col in df_salary.columns:
        df_salary[col + '_Encoded'] =
label_encoder.fit_transform(df_salary[col]) print(f"\nDataFrame
        head after Label Encoding '{col}':")
        print(df_salary[[col, col + '_Encoded']].head())
    else: print(f"\nWarning: '{col}' column not found for label
encoding.")

# Drop original categorical columns if desired (optional)
df_salary.drop(columns=[col for col in categorical_cols_to_encode if
col in df_salary.columns], inplace=True)
```

```
print("\nCleaned DataFrame head ready for analysis:")
print(df_salary.head())
print("\nCleaned DataFrame info ready for analysis:")
df_salary.info()
```

```
Creating dummy 'employee_salary_data.csv' for demonstration.
Original DataFrame head:
   EmployeeID   Age  Gender Department Job_Location  Years_Experience
\
0           1  33.0    Male      Sales     New York              10.0

1           2  43.0    Male  Marketing       London              11.0

2           3  43.0  Female         IT        Paris               9.0

3           4  51.0  Female         HR       London              15.0

4           5  59.0    Male    Finance       London               NaN


     Salary  Performance_Score
0  36918.44                  1
1  39934.64                  2
2  55039.23                  4
3  57416.14                  2
4  49975.46                  4

Original DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   EmployeeID         100 non-null    int64
 1   Age                95 non-null     float64
 2   Gender             100 non-null    object
 3   Department         95 non-null     object
 4   Job_Location       100 non-null    object
 5   Years_Experience   95 non-null     float64
 6   Salary             95 non-null     float64
 7   Performance_Score  100 non-null    int64
dtypes: float64(3), int64(2), object(3)
memory usage: 6.4+ KB

DataFrame after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
```

```
  0   EmployeeID      100 non-null    int64
  1   Age   100 non-null    float64
  2   Gender     100 non-null    object
  3   Department      100 non-null    object
  4   Job_Location    100 non-null    object
  5   Years_Experience 100 non-null   float64
  6   Salary     100 non-null    float64
  7   Performance_Score 100 non-null   int64 dtypes: float64(3),
 int64(2), object(3) memory usage: 6.4+ KB
Number of outliers detected in 'Salary': 1
Outliers in Salary:
     EmployeeID
     Salary 5   6
150000.0

DataFrame shape after removing Salary outliers: (99, 8)
DataFrame head after converting 'Department' to lowercase:
  Department
0         sales
1         marketing
2         it
3         hr
4         finance
DataFrame head after Label Encoding  'Job_Location':
  Job_Location Job_Location_Encoded
0        New York     2
1        London 1
2        Paris  3
3        London 1
4        London 1

DataFrame head after Label Encoding
                                    'Department':
 Department Department_Encoded
0         sales 4
1         marketing   3
2         it    2
3         hr    1
4         finance     0
Cleaned DataFrame head ready for analysis:
```

```
    EmployeeID Age Gender Years_Experience       Salary 0
Performance_Score \                                     1
              1 33.0  Male           10.000000 36918.44 1
                                                        2
              2 43.0  Male           11.000000 39934.64 2

             3 43.0 Female            9.000000 55039.23
```

```
4
3                 51.0 Female        15.000000 57416.14
                4
2
4                 59.0    Male       14.631579 49975.46
                5
4

   Job_Location_Encoded Department_Encoded
0     2     4
1     1     3 2   3     2 3   1     1
4                       1                    0

Cleaned DataFrame info ready for analysis:
<class
'pandas.core.frame.DataFrame'>
Index: 99 entries, 0 to
99
Data   columns   (total   8
columns):
 #   Column       Non-Null CountDtype

     --------
-----                                 ---------
   -----------------


 0   EmployeeID      99 non-nullint64
1 Age                      99 non-null
                              float64
  2   Gender     99 non-null      object
  3   Years_Experience      99 non-null      float64
  4   Salary     99 non-null      float64
  5   Performance_Score     99 non-null      int64
  6   Job_Location_Encoded 99 non-null       int64
  7   Department_Encoded    99 non-null      int64
 dtypes: float64(3), int64(4), object(1) memory usage:
 7.0+ KB
```